Hitesh Rathore (HR) Notes

LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

</> HTML + INTERNAL CSS </>

ACTUAL BROWSER INTERNAL WORKING

★ PHASE 0 — URL → RAW BYTES PIPELINE ★

USER ACTION

User types in browser https://example.com and presses ENTER

FIRST IMPORTANT MENTAL MODEL

Browser is NOT: HTML → screen

Browser is actually: Distributed Systems + Networking Engine + Parser Engine + Layout Engine + Graphics Engine + GPU Pipeline + JavaScript Runtime + Multi-process OS Application

BROWSER ARCHITECTURE (MULTI-PROCESS)

- Browser Process
- Renderer Process (Tab)
- GPU Process
- Network Process
- Utility Processes
- Extension Processes

STUDENT DOUBT "Why separate "processes?"

- ✓ If webpage crashes (while(true){}), only tab freezes.
- ✗ NOT entire browser.
- ✓ Security sandboxing: Websites are untrusted code. Browser isolates them.

① URL PARSING

Browser parses the URL into components.

```
URLObject {
  schema: HTTPS,
  host: "example.com",
  port: 443,
  path: "/"
}
```

Structured object for faster internal access.

② CACHE CHECK

Browser checks multiple caches before request.

- ✓ Memory Cache
- ✓ Disk Cache
- ✓ Service Worker Cache
- ✓ HTTP Cache
- ✓ Preload Cache

STUDENT DOUBT "Why check cache first?" Because network is EXPENSIVE! Cache can avoid ALL that.

③ DNS RESOLUTION

Browser asks DNS server for IP address.

DNS Server

example.com + 93.184.216.34

STUDENT DOUBT "Why not use domain names directly?" Routers use numeric IPs. Faster & standardized. Like house address system.

④ TCP CONNECTION

Browser establishes TCP connection.

TCP PURPOSE: Reliable communication. Guarantees:

- ✓ Packet ordering
- ✓ Retransmission
- ✓ Error handling
- ✓ Reliable byte stream

IMPORTANT: HTTP is NOT responsible for reliability. TCP handles that.

⑤ TLS HANDSHAKE (HTTPS)

Encrypted connection is established.

TLS

During TLS:

- ✓ Exchange certificates
- ✓ Verify authenticity
- ✓ Negotiate algorithms
- ✓ Generate session keys

STUDENT DOUBT "Why not do expensive TLS every request?" Because browsers use:

- TLS session reuse
- Connection pooling
- HTTP keep-alive
- HTTP/2 multiplexing

Huge optimization!

⑥ HTTP REQUEST CREATION

Browser creates HTTP request (text format).

```
GET / HTTP/1.1
Host: example.com
User-Agent: Chrome
Accept: text/html
...
```

⑦ REQUEST → RAW BYTES

Request is converted into binary bytes.

```
01000111 01000101
01010100 00100000
00101111 00100000
01001000 01010100
01010100 01010000
...
```

⑧ PACKETS

Bytes are split into network packets.

Request Bytes → Packet 1, Packet 2, Packet 3 … Packet N

STUDENT DOUBT "Why packets?" Internet is distributed. Packets allow:

- ✓ Routing flexibility
- ✓ Retransmission
- ✓ Congestion handling
- ✓ Scalable networking

⑨ SERVER RECEIVES REQUEST

Server reconstructs packets and processes the request.

Packet 1, Packet 2, Packet 3 … Packet N →

Handled by: NGINX, node (Node.js), Apache, Go, Rust etc.

Server generates HTML response.

⑩ RESPONSE SENT BACK

Server converts response into bytes, packets and sends back.

Bytes → Packets → Internet

⑪ RAW BYTES ARRIVE

Browser receives raw byte stream.

`00111100 01101000 0110000 0110 01101100 …`

NOT HTML YET!

MOST IMPORTANT REALITY

Browser first receives: RAW BYTES

NOT: DOM | HTML objects | CSS objects

Everything starts from BYTES.

⑫ PARSING & RENDERING PIPELINE (BROWSER INTERNAL WORKFLOW)

HTML Parser → DOM Tree → CSS Parser (Internal CSS) → CSSOM Tree → DOM + CSSOM = Render Tree → Layout (Geometry) → Paint (Raster) → Composite (GPU) → Final Pixels On Screen

REALITY CHECK MEME

USER THINKS: "Browser bas HTML dikhata hoga"

VS

BROWSE REALITY: "Bro, I run half the computer!"

GOLDEN TAKEAWAY

From URL enter to final pixels, browser goes through:

Network Systems + Security + Protocols + Parsing + Rendering + GPU

All starting from RAW BYTES.

PHASE 1 — BYTE DECODING + STREAMING HTML PARSER

(REAL BROWSER INTERNAL FLOW)

WHERE WE LEFT OFF (End of Phase 0)

Browser received RAW NETWORK BYTES from the server.

Browser receives a STREAM of bytes (not full file at once)

Packet 1 → Packet 2 → Packet 2 → . . . Packet N

MOST IMPORTANT REALITY

Browser first receives: RAW BYTES

NOT: DOM, HTML objects, or CSS objects!

Everything starts from BYTES.

STEP 14–15 ENCODING DETECTION (How browser decides how to read bytes)

① PRIORITY 1 – BOM: Check initial bytes for BOM. Example: EF BB BF → UTF-8 BOM (EF | BB | BF)

② PRIORITY 2 – HTTP HEADER: Server may send header Content-Type: text/html; charset=UTF-8

③ PRIORITY 3 – META TAG: Browser may find later in HTML `<meta charset="UTF-8">`

④ PRIORITY 4 – DEFAULT / HEURISTIC: If nothing specified, use UTF-8 (modern default) or legacy heuristics.

⑬ RESPONSE BODY STARTS ARRIVING

Browser receives bytes incrementally (TCP stream). Does NOT wait for full HTML before processing.

Why not wait full HTML?

- → To reduce First Paint latency
- → Progressive rendering.
- → Early resource discovery.

⑭ ENCODING DETECTION STARTS

Browser must determine character encoding. Bytes alone are meaningless.

Example: 01000001 can be different characters in different encodings.

⑯ BYTE DECODER STARTS

3C 68 74 → < h t

Now browser knows how to map bytes → characters.

Example:

```
0x3C → '<'
0x68 → 'h'
0x74 → 't'
```

⑰ STREAMING DECODER

Decoder works in streaming manner, not waiting for full response.

Incoming bytes ↓ Decode immediately ↓ Send characters forward

⑱ HTML TOKENIZER STARTS

Converts characters into HTML tokens.

Example Input: `<h1>Hello</h1>`

Tokens Output:

- START_TAG(h1)
- TEXT("Hello")
- END_TAG(h1)

⑲ TOKENIZER = STATE MACHINE

Tokenizer changes states based on context.

Example States:

- DATA STATE
- TAG OPEN STATE
- ATTRIBUTE STATE
- COMMENT STATE
- SCRIPT DATA STATE

Why states needed? Same character can have different meaning in different context.

STUDENT DOUBT Why separate Tokenizer and Parser?

- ✓ Tokenizer → Lexical Analysis (characters → tokens)
- ✓ Parser → Structural Interpretation (tokens → DOM Tree)

Same idea used in: Compilers, Interpreters, Programming Languages.

TOKENIZER STATE MACHINE (EXAMPLE FLOW)

DATA STATE ↓ TAG OPEN STATE ↓ ATTRIBUTE STATE ↓ VALUE STATE ↓ COMMENT STATE ↓ SCIIPT DATA STATE ↓ SCRIPT DATA STATE ↓ … and many more

Transitions depend on current state + input char.

⑳ HTML PARSER STARTS

Parser consumes tokens and understands structure.

Consumes Tokens:

```
START_TAG(h1)
TEXT("Hello")
END_TAG(h1)
```

㉑ DOM NODE ALLOCATION

Browser allocates internal objects (not JS objects initially).

Examples:

- HTMLHeadingElement
- TextNode

Implemented in C++ / Rust with optimized memory layouts, arenas, pointers.

㉒ TREE CONSTRUCTION

Parser links nodes to build DOM tree.

Example DOM Tree:

```
Document
 └ html
    └ body
       └ h1
          └ Text("Hello")
```

㉓ PARSER CONTINUES WHILE NETWORK ACTIVE

Networking, decoding, tokenizing, parsing all run simultaneously.

This is true streaming pipeline. Everything is incremental and overlapped.

SAMPLE HTML & RESULT

Sample HTML:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello</h1>
  </body>
</html>
```

Resulting DOM Tree (conceptual):

```
Document
 └ html
    └ body
       └ h1
          └ Text("Hello")
```

IMPORTANT ★

Encoding detection is a complex subsystem because old websites used many encodings:

- Shift-JIS
- ISO-8859-1
- Windows-1252
- GBK
- Big5
- … etc

KEY TAKEAWAY

Browser is a highly optimized streaming machine. It never waits unnecessarily.

Bytes → Characters → Tokens → Nodes → Tree → Render.

Everything is incremental.

REAL TIME SIMULTANEOUS PIPELINE (What actually happens in browser)

NETWORK (TCP STREAM) → 1. BYTES ARRIVE → 2. ENCODING DETECTION → 3. BYTE DECODER → 4. TOKENIZER → 5. PARSER → 6. DOM CONSTRUCTION → RENDERING PIPELINE (Layout → Paint → Composite) → PIXELS ON SCREEN (Visible to User)

RESOURCE DISCOVERY (Images, CSS, JS, etc.) happens during parsing → New requests start in parallel

WHY THIS ARCHITECTURE MATTERS?

- ULTRA FAST PERFORMANCE: Streaming + parallel processing = webpages load insanely fast.
- LOW MEMORY USAGE: Browsers don't wait for the full file; they process as bytes arrive.
- BETTER USER EXPERIENCE: Content appears quickly while the rest continues loading in background.
- ROBUST & RESILIENT: Works even on slow networks, incomplete data, or errors.
- SCALABLE BY DESIGN: Handles millions of resources, connections and tabs efficiently.
- THE FOUNDATION OF MODERN WEB: This pipeline powers every website, web app, and online experience.

PHASE 2 — INTERNAL CSS DISCOVERY + CSS PARSER ENGINE

From <style> Encountered → Stylesheet Object Created

WHERE WE ARE (End of Phase 1)

Browser has built part of the DOM (Streaming) → DOM Tree (Partial)

MOST IMPORTANT MENTAL MODEL

HTML Parsing Creates: DOCUMENT STRUCTURE

CSS Parsing Creates: STYLE RULE SYSTEM

Later Browser Combines BOTH for Rendering

㉖ HTML PARSER ENCOUNTERS <style>

Example HTML:

```
<head>
  <style>
    h1 {
      color: blue;
    }
  </style>
</head>
```

Browser does NOT treat `<style>` content as HTML.

Inside `<style>`, content is handled as CSS, NOT DOM.

㉗ HTML PARSER HANDS CONTROL TO CSS ENGINE

HTML parser extracts raw text inside `<style>` and sends to CSS engine.

Raw CSS Text Extracted:

```
h1 {
  color: blue;
}
```

HTML Parser ⟶ CSS Engine (Division of Responsibilities)

㉘ PARSER BLOCKING BEHAVIOR

Internal CSS can affect rendering.

Browser may pause certain rendering work until this CSS is parsed.

Example Problem: Render `<h1>Hello</h1>` before CSS says `h1 { display: none; }` = Wrong Flicker!

㉙ CSS TOKENIZER STARTS

Input (CSS Text):

```
h1 {
  color: blue;
}
```

Converts characters into CSS TOKENS

Tokens Output:

```
1. IDENT(h1)
2. LBRACE({)
3. IDENT(color)
4. COLON(:)
5. IDENT(blue)
6. SEMICOLON(;)
7. RBRACE(})
```

㉚ CSS PARSER STARTS

Parser consumes tokens and understands structure.

Example:

```
h1 {
  color: blue;
}
```

Understood As: Selector: h1; Declarations: color = blue

㉛ SELECTOR PARSING

Parses "h1" as a selector structure.

Internal Representation:

```
SelectorObject {
  type: TAG_SELECTOR,
  value: H1
}
```

Why? Efficient matching against thousands or millions of DOM nodes later.

㉜ DECLARATION PARSING

Parses property and value separation.

Input: `color: blue;`

Separated As: Property → color; Value → blue

Property names converted to internal IDs / enums.

Example: PROPERTY_COLOR = 17

㉝ VALUE PARSING

Parses value into optimized internal format.

Input Value: blue

Internal Representation: RGB(0, 0, 255) or 0x0000FF

Another Example: `font-size: 40px;` → value = 40, unit = PX

㉞ STYLESHEET OBJECT CREATION

Creates internal stylesheet structures.

Conceptual Structure:

```
Stylesheet
 └ Rule
    ├ Selector: h1
    └ Declarations
       └ color = blue
```

This is the core output of internal CSS parsing.

㉟ CSSOM-LIKE STRUCTURES EMERGE

Now browser has: DOM Tree (Structure) + Stylesheet Rules (Style System)

People call this CSSOM simplistically, but internally engines use highly optimized rule systems (engine dependent).

㊱ RESOURCE DISCOVERY CONTINUES (PARALLEL)

While CSS parsing continues, other work ALSO continues:

- ✓ Networking
- ✓ HTML Parsing
- ✓ DOM Construction
- ✓ Image Discovery
- ✓ Script Loading
- ✓ Preload Scanner
- ✓ etc...

Everything is highly pipelined & parallel.

INLINE CSS vs INTERNAL CSS

INLINE CSS: `<h1 style="color: blue">Hello</h1>` — Element `<h1>` → Direct Style Attachment. Style attached directly to element.

INTERNAL CSS:

```
<style>
  h1 { color: blue; }
</style>
```

Stylesheet Rules (Shared) ↓ Selector Matching ↓ Apply to DOM Nodes

Applied dynamically via selector matching.

REAL PIPELINE (SIMULTANEOUS WORKFLOW)

1 Network Stream (101001 010110 101101) → 2 Byte Decoder (0x3C 0x68 0x74 0x6D) → 3 HTML Tokenizer → 4 HTML Parser → 5 DOM Construction → 6 `<style>` Encountered → 7 CSS Tokenizer → 8 CSS Parser → 9 Stylesheet Object Created → 10 Style Resolution (Later Phase)

ALL RUNNING IN PARALLEL (Not strictly one after another)

STUDENT DOUBT Why not apply styles during parsing? Because browser may not know yet:

- ✗ Future DOM nodes
- ✗ Future stylesheets
- ✗ Cascade & specificity conflicts
- ✗ Inheritance dependencies

✓ Thus parsing is separated from style resolution.

INTERNAL CSS PARSING BUILDS THE STYLE RULE SYSTEM, WHICH LATER COMBINES WITH DOM TO PRODUCE FINAL STYLES!

PHASE 3 – STYLE ENGINE + SELECTOR MATCHING

REAL BROWSER STYLE RESOLUTION PIPELINE

INPUT AVAILABLE TO STYLE ENGINE

DOM Tree: Document → html → body → h1 → "Hello"

Stylesheet Rules:

```
h1 {
  color: blue;
}
.title {
  color: green;
}
#mainTitle {
  color: red;
}
```

STYLE ENGINE'S JOB

DOM + CSS Rules ↓ Final Computed Styles

REAL PIPELINE (PHASE 3 OVERVIEW)

Selector Matching → Cascade Resolution → Inheritance Resolution → Computed Style Objects

WHAT STYLE ENGINE PRODUCES

Browser now knows HOW elements should look.

NOT where they are, not layout, not painting, not pixels.

㊲ STYLE RESOLUTION STARTS

Example: HTML `<h1>Hello</h1>`; CSS `h1 { color: blue; }`

Browser asks: Does this rule apply to this DOM node?

This is called: SELECTOR MATCHING

㊳ SELECTOR MATCHING ENGINE

Browser takes selector: h1. Checks DOM nodes. Conceptually:

DOM Node	Selector	Match?
`<h1>Hello</h1>`	h1	✓ YES

Why is selector matching expensive? Because webpages may contain:

- millions of DOM nodes
- thousands of CSS rules

Naive matching would be VERY slow.

㊴ SIMPLE SELECTORS ARE FAST

Examples: h1 · .title · #header

Why fast? Because browser can optimize lookups heavily.

Example (conceptual): Class Map (Index)

Key	Nodes
"title"	[node1, node2, node3]
"button"	[node10, node11]
"card"	[node20, node21, node22]

Modern browsers maintain internal indexing systems. NOT brute-force scanning every time.

㊵ COMPLEX SELECTORS ARE HARDER

Example Selector: `.sidebar .menu ul li a span.title`

Why expensive? Browser must evaluate:

- ancestor relationships
- parent chains
- nested structures

Browser may need traversal like: span.title ↑ a ↑ li ↑ ul ↑ .menu ↑ .sidebar

㊳A RIGHT TO LEFT MATCHING (VERY IMPORTANT)

CSS selectors are matched RIGHT TO LEFT, not left-to-right.

Why right-to-left?

Suppose selector: `.sidebar .menu span`

Browser starts from: span (actual target element)

Then walks upward checking ancestors.

Right-to-Left Matching: span (start) ↑ .sidebar (ancestor) ↑ .sidebar (ancestor)

✓ Match Found!

㊶ STYLE RULE COLLECTION

Browser collects ALL matching rules for the element.

Example Rules:

```
h1 { color: blue; }
.title { color: green; }
#mainTitle { color: red; }
```

Element: `<h1 id="mainTitle" class="title">Hello</h1>`

Matched (Rule 1) · Matched (Rule 2) · Matched (Rule 3)

Multiple rules may apply simultaneously. Now browser must resolve conflicts.

㊷ CASCADE RESOLUTION

Browser determines: Which rule wins?

Browser considers (in order):

1. Origin (user agent / user / author)
2. Importance (!important)
3. Specificity
4. Source Order (later wins if tie)

Example:

Rule	Specificity	Source Order
h1 { color: blue; }	0,0,1	1
.title { color: green; }	0,1,0	2
#mainTitle { color: red; }	1,0,0	3

Result: red wins (highest specificity)

㊸ SPECIFICITY CALCULATION

Browser calculates specificity numerically.

Conceptual Weights:

- ID selector = (1,0,0)
- Class selector = (0,1,0)
- Tag selector = (0,0,1)

Examples:

Selector	Specificity
h1	(0,0,1)
.title	(0,1,0)
#mainTitle	(1,0,0)
h1.title	(0,1,1)
div#main .title	(1,1,0)

Fast numeric comparison.

㊹ INHERITANCE RESOLUTION

Some properties are inherited from parent.

Example:

```
body {
  color: blue;
  font-family: Arial;
  font-size: 16px;
}
```

Child `<p>Hello</p>` Inherits: color: blue; font-family: Arial; font-size: 16px

Do NOT inherit (examples): margin, border, padding, width, height, background, display, …

Why inheritance exists? Reduces duplication. Otherwise every element must be styled individually.

㊺ DEFAULT USER AGENT STYLES

Even without CSS, browser applies default styles.

Example (User Agent Stylesheet):

```
h1 {
  display: block;
  font-size: 2em;
  font-weight: bold;
  margin-block: 0.67em;
}
```

Final style result combines:

- User Agent Styles
- Author Styles
- Inline Styles
- Inherited Values

㊻ COMPUTED STYLE CREATION

Browser creates COMPUTED STYLE OBJECT. Purpose: produce final resolved values.

Example ComputedStyle:

```
ComputedStyle {
  color: rgb(255, 0, 0);
  font-size: 32px;
  font-family: Arial;
  font-weight: bold;
  display: block;
  margin-block: 0.67em;
  line-height: normal;
  /* ... many more ... */
}
```

Important:

- ✓ Cascade resolved
- ✓ Inheritance resolved
- ✓ Defaults resolved
- ✓ Units mostly finalized

㊼ STYLE SHARING OPTIMIZATION

Suppose page has many identical nodes:

```
<li class="item">A</li>
<li class="item">B</li>
<li class="item">C</li>
```

All have same computed style? YES

Share same ComputedStyle object (not recalculated)

Why important? Large webpages may contain thousands of visually similar nodes.

Saves CPU + Memory.

㊽ STYLE INVALIDATION SYSTEM

Suppose JS changes: `element.className = "active";`

Browser must determine: Which elements require style recalculation?

✗ Recalculating entire page styles every time is too expensive!

Thus browser uses INVALIDATION SYSTEMS to limit recalculation scope.

Only affected elements (and their descendants) are recalculated.

㊾ STYLE ENGINE OUTPUT

At end of style phase browser now has:

DOM Nodes + Computed Styles (node1 → style obj, node2 → style obj, node3 → style obj …)

But still:

- ✗ no geometry
- ✗ no layout positions
- ✗ no painting
- ✗ no pixels

Browser now knows HOW elements should look, but not WHERE they appear. That comes next.

WHY BROWSER ENGINES ARE SO COMPLEX?

Because they must optimize simultaneously for:

Huge DOM trees (millions of nodes) · Dynamic JavaScript changes · Animations & Transitions · Scrolling & Interactivity · Low Memory Usage · High FPS Rendering

All while keeping the UI smooth and fast!

PHASE 4 — RENDER TREE + LAYOUT ENGINE

WHERE STYLES BECOME REAL VISUAL GEOMETRY

INPUT TO PHASE 4

Browser already has:

1. DOM Tree: Document → html → body → "Hello"
2. Computed Styles:

```
ComputedStyle {
  color: blue;
  font-size: 40px;
  display: block;
}
```

BUT BROWSER STILL DOES NOT KNOW:

- ✗ exact x / y position
- ✗ width / height
- ✗ line wrapping
- ✗ overflow
- ✗ glyph positions
- ✗ pixels

PHASE 4 TRANSFORMS:

DOM + Computed Styles ↓ Render Tree ↓ Layout / Reflow ↓ Exact Geometry

WHAT CHANGES IN THIS PHASE?

Before: Browser knows HOW elements should look

After: Browser knows WHERE elements should appear

㊿ RENDER TREE CREATION STARTS

Browser combines: DOM + Computed Styles to create: RENDER TREE

IMPORTANT: DOM ≠ Render Tree. They are DIFFERENT structures.

(51) NON-VISUAL DOM NODES MAY BE EXCLUDED

Example:

```
<head>
  <meta>
  <script>
  ...
</head>
```

These usually do NOT appear in Render Tree because they do not visually render.

STUDENT DOUBT "Why not render directly from DOM?" Because rendering system needs:

- ✓ geometry logic
- ✓ line formatting
- ✓ paint logic
- ✓ visual
- ✓ layout algorithms
- ✓ optimizations

DOM alone is insufficient.

(52) DISPLAY:NONE SPECIAL CASE

Suppose: `display: none;`

IMPORTANT: Element may exist in: DOM ✓ BUT NOT in: Render Tree ✗

WHY? Because `display:none` means: do not visually participate

HUGE DIFFERENCE

Structure	Exists?
DOM	✓
Render Tree	✗

(53) RENDER OBJECT TYPES

Browser creates different render/layout objects. Examples conceptually:

- RenderBlock
- RenderInline
- RenderText
- RenderImage
- FlexLayoutObject
- GridLayoutObject
- …

IMPORTANT: Modern engines use highly optimized internal classes.

Examples: Blink → LayoutObject/LayoutNG; WebKit → RenderObject; Gecko → Frame Tree

Architecture concept is similar.

(54) RENDER TREE RELATIONSHIPS

Browser links render objects together. Conceptually:

RenderBlock(body) ↓ RenderBlock(h1) ↓ RenderText("Hello")

IMPORTANT: Render Tree often mirrors DOM structure partially, but not always exactly.

WHY RENDER TREE ≠ DOM TREE?

Because rendering behavior may differ. Examples:

- display: contents → element disappears visually but children remain.
- Pseudo-elements (::before, ::after) appear visually without DOM nodes.
- Anonymous boxes → browser creates internal layout containers.
- List markers → `<li>` creates marker boxes internally.

STUDENT DOUBT "Why not exactly same as DOM?" Because visual structure can change independently of DOM structure.

(55) LAYOUT PHASE STARTS

Now browser begins: LAYOUT

also called:

- Reflow
- Formatting
- Geometry Calculation

MOST IMPORTANT PURPOSE: Determine exact positions and sizes for render objects. This is where browser computes:

- ✓ x position
- ✓ line wrapping
- ✓ y position
- ✓ baselines
- ✓ width
- ✓ overflow areas
- ✓ height
- ✓ and more

(56) WHY LAYOUT IS SEPARATE

Render Tree creation only answers: "What visual objects exist?"

Layout answers: "Exactly where and how large?"

Example: Render Tree knows: h1 exists; font-size = 40px

BUT layout must still compute:

- ✗ actual width
- ✗ actual height
- ✗ line breaks
- ✗ final coordinates

(57) LAYOUT DEPENDENCIES

Geometry depends on MANY things.

Example: `<h1>Hello</h1>`

Rendered width depends on:

- font metrics
- font rendering system
- viewport width
- writing mode
- zoom level
- parent constraints
- device DPI
- and more

IMPORTANT: Layout is computationally expensive.

(58) BLOCK FORMATTING CONTEXT

Suppose: `display: block;`

Browser applies block layout rules.

Example: div / div / div

- Block elements stack vertically.
- Take available width.

(59) INLINE FORMATTING CONTEXT

Suppose text: "Hello world"

Browser must compute:

- glyph positions
- kerning
- word wrapping
- ligatures
- line boxes
- bidi text
- baselines
- and more

IMPORTANT: Text layout is EXTREMELY complicated. Because browser handles:

- ✓ Unicode
- ✓ ligatures
- ✓ font shaping
- ✓ international scripts
- ✓ emoji
- ✓ and much more

(60) FLEXBOX / GRID ALGORITHMS

Modern layout systems are complex.

Example: `display: flex;`

Browser must calculate:

- ✓ flex grow / shrink
- ✓ alignment
- ✓ available space distribution
- ✓ min / max constraints

GRID layout is even more complex.

(61) LAYOUT OUTPUT

At end of layout browser finally has: EXACT GEOMETRY

Example (conceptually):

```
RenderBlock(h1)
  x = 20
  y = 100
  width = 180
  height = 52
```

IMPORTANT: NOW browser knows exact physical geometry for rendering.

(62) LAYOUT INVALIDATION SYSTEM

VERY important optimization.

Suppose JS changes: `element.style.width = "500px";` — Browser may need: ☑ layout recalculation

BUT if only: `element.style.color = "red";` — then: ✗ layout may NOT be needed because geometry unchanged.

THIS is why browsers separate phases. Huge optimization!

REAL PIPELINE NOW

DOM Tree → Computed Styles → Render Tree Creation → Layout / Reflow → Exact Geometry

MOST IMPORTANT REALITY OF PHASE 4

At end of this phase, browser has converted abstract styles into concrete geometry.

READY FOR PAINTING!

PHASE 5 — PAINT + COMPOSITING + GPU PIPELINE

(WHERE BROWSER FINALLY PRODUCES REAL PIXELS)

(63) PAINT PHASE STARTS

Browser now traverses render/layout objects.

Purpose: Generate PAINT INSTRUCTIONS

Conceptually:

- Draw blue text
- Fill black rectangle
- Draw border
- Draw shadow

IMPORTANT: Paint phase is NOT direct GPU drawing yet. Initially browser creates paint commands / display lists.

(64) DISPLAY LIST CREATION

VERY IMPORTANT INTERNAL CONCEPT. Browser usually creates something conceptually like: DISPLAY LIST

Example:

1. Fill background rectangle
2. Draw border
3. Draw text
4. Draw shadow
5. Draw image

WHY DISPLAY LISTS? Because browser wants separation between: WHAT to draw and HOW pixels are rasterized

STUDENT DOUBT "Why not directly draw immediately?" Because browser needs: batching, optimization, GPU coordination, caching, partial repaint support. Display lists make this easier.

(65) PAINTING TEXT

Suppose:

```
color: blue;
font-size: 40px;
```

Browser now prepares text painting operations.

IMPORTANT: Browser still does NOT paint characters directly. Internally browser works with:

- glyphs
- font rasterization
- text shaping
- subpixel rendering

WHY? Because text rendering is EXTREMELY advanced. Example:

- ligatures
- emoji
- Arabic shaping
- Hindi scripts
- Unicode combining characters

must render correctly.

(66) PAINTING BACKGROUNDS &

Browser now paints:

- ✓ background colors
- ✓ gradients
- ✓ borders
- ✓ shadows
- ✓ outlines

Example:

```
background: black;
border: 2px solid red;
```

Paint commands conceptually: FillRect() · DrawBorder()

(67) REPAINT VS REFLOW

VERY IMPORTANT PERFORMANCE TOPIC.

Suppose: `element.style.color = "red";`

Browser may need: ☑ repaint

BUT: ✗ layout/reflow may NOT be needed because geometry unchanged.

Another Example: `element.style.width = "500px";`

Now browser may need: ☑ layout ☑ repaint because geometry changed.

HUGE PERFORMANCE DIFFERENCE

Operation	Cost
Paint only	cheaper
Layout + Paint	expensive

(68) LAYER SYSTEM STARTS

Modern browsers often create: COMPOSITING LAYERS

Conceptually: Layer 1 Background · Layer 2 Navbar · Layer 3 Content · Layer 4 Video

IMPORTANT: Browser does NOT always paint entire page as one flat bitmap. Instead browser may separate:

- ✓ scrolling content
- ✓ fixed headers
- ✓ videos
- ✓ transformed elements
- ✓ opacity animations

into layers.

STUDENT DOUBT "Why layers?" Because browser wants: GPU acceleration, efficient scrolling, efficient animations, partial updates without repainting whole page.

(69) COMPOSITING PHASE STARTS

NOW browser combines layers.

IMPORTANT: Paint phase generated content. Composite phase assembles final frame.

MOST IMPORTANT DIFFERENCE: Paint — Create pixels for elements; Composite — Combine already-painted layers efficiently

Example: Suppose element uses: `transform: translateX(100px);`

Browser may: ✗ avoid repaint ✗ avoid layout ⇨ and simply: ☑ move compositing ☑ layer using GPU.

(70) GPU PROCESS INVOLVEMENT

Modern browsers usually have: GPU PROCESS separate from renderer.

WHY? Because GPU specialized for: parallel graphics operations

Browser uploads layer textures to GPU.

Conceptually: Painted Layer → GPU Texture → Composite on GPU

(71) RASTERIZATION

VERY IMPORTANT. Display lists eventually become: actual pixels through: RASTERIZATION

Rasterization means: vector instructions ↓ pixel bitmap

Example: DrawRect() becomes: actual colored pixels in memory

(72) FRAME GENERATION

Browser now produces: FRAME. Conceptually: Final composed image for current moment

IMPORTANT: Modern browsers try rendering at: 60 FPS meaning: ~16.6ms per frame for ALL: JS, style, layout, paint, composite combined. Extremely hard engineering problem.

(73) VSYNC SYNCHRONIZATION

Browser coordinates with monitor refresh timing. Purpose: Avoid screen tearing. Browser schedules frame presentation carefully. 60Hz

(74) FINAL SCREEN OUTPUT

NOW finally: photons emitted from screen. User sees: Hello

COMPLETE PIPELINE SUMMARY

RAW BYTES ↓ Byte Decoding ↓ HTML Tokenization ↓ HTML Parsing ↓ DOM Tree ↓ CSS Parsing ↓ Stylesheet Rules ↓ Selector Matching ↓ Computed Styles ↓ Render Tree ↓ Layout ↓ Paint ↓ Composite ↓ GPU Rasterization ↓ Final Pixels on Screen

EXTERNAL CSS — ONLY WHAT'S DIFFERENT

External CSS is written in a separate file with .css extension and linked to HTML using `<link>` tag.

1. WHERE IT IS WRITTEN (DIFFERENCE)

External CSS lives in a separate .css file outside the HTML.

index.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>External CSS Example</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>Hello, External CSS!</h1>
    <p>This paragraph uses external CSS.</p>
  </body>
</html>
```

styles.css

```
/* This is an external CSS file */
body {
  background-color: #0d1117;
  color: white;
}
h1 {
  color: #58a6ff;
}
p {
  color: #c9d1d9;
}
```

2. HOW BROWSER LOADS IT (DIFFERENCE)

After parsing HTML, when browser sees `<link rel="stylesheet" href="...">` it:

Parse HTML → Find `<link>` to external CSS → Fetch CSS file (separate request) → Parse CSS & add rules to Stylesheet

This may involve a network request (unless cached).

3. SCOPE (DIFFERENCE)

Applies to all HTML pages that link this CSS file. One .css file can style many pages.

Only applies to pages that include it. Pages not linking it are unaffected.

4. CACHING ADVANTAGE (DIFFERENCE)

Browsers cache external CSS files aggressively. So subsequent pages load faster.

Better performance for multi-page websites.

5. RENDERING BEHAVIOR (SAME AS INTERNAL)

CSS rules from external file are applied → Computed Styles are created → Render Tree is built → Layout is done → Paint, Composite, GPU Pipeline... → Final pixels on screen

SUMMARY

External CSS is almost the same as Internal CSS in how it works.

The only real differences are:

- ✓ Written in a separate .css file
- ✓ Linked using `<link>` tag
- ✓ May involve network request
- ✓ Can be reused across many pages
- ✓ Better caching and performance
- ✓ Same scope, priority & rendering behavior

```
<head>
  <link rel="stylesheet"
        href="styles.css">
</head>
```

COMPARISON: INLINE vs INTERNAL vs EXTERNAL CSS

WHICH IS FASTER?

CSS TYPE	SPEED (INITIAL LOAD)	WHY?	REAL BROWSER BEHAVIOR
INLINE CSS (style attribute) `<h1 style="color:red">Hello</h1>`	FASTEST (small scope)	Style already attached to element No selector matching needed No separate CSS file No stylesheet traversal	`<h1 style="...">` DOM Node → Direct style attachment → Computed Style
INTERNAL CSS (`<style>` in `<head>`) `<style> h1 { color: red; } </style>`	VERY FAST	CSS available inside same HTML document Browser parses `<style>` before rendering No network request CSSOM built immediately	HTML Parse → CSS Parse → CSSOM → Render Tree
EXTERNAL CSS (linked .css file) `<link rel="stylesheet" href="style.css">`	SLIGHTLY SLOWER INITIALLY	Needs separate network request Download CSS file first Parse CSS Build CSSOM	HTML Parse → Find `<link>` → Download CSS → Parse CSS → CSSOM

REAL WORLD PERFORMANCE SUMMARY

INITIAL RAW SPEED (FIRST LOAD): Inline CSS Fastest > Internal CSS Very Fast > External CSS Slightly Slower

REAL WORLD LARGE APP PERFORMANCE (AFTER CACHING & AT SCALE): External CSS Best Overall > Internal CSS Good > Inline CSS Not Ideal

WHY EXTERNAL CSS IS BEST FOR REAL PROJECTS?

- ✓ Browser can cache the stylesheet
- ✓ Reuse across multiple pages
- ✓ Better for maintainability
- ✓ Smaller HTML size
- ✓ Parallel downloads
- ✓ Optimized selector matching
- ✓ Works great with HTTP/2, HTTP/3
- ✓ Used by all modern frameworks

KEY TAKEAWAYS

- Inline is fastest only for very small cases.
- Internal is very fast and better than inline for maintainability.
- External is slightly slower initially but becomes the best choice in real-world applications.
- Actual performance difference is tiny in modern browsers.

WHAT REALLY AFFECTS PERFORMANCE?

- ✗ Huge DOM
- ✗ Bad JavaScript
- ✗ Layout thrashing
- ✗ Large images
- ✗ Unnecessary reflows
- ✗ Animations
- ✗ Heavy frameworks

NOT whether CSS is inline, internal or external.

★ FINAL VERDICT: Use EXTERNAL CSS for real projects. It scales better, caches better, and is optimized by browsers.

GPU PIPELINE IN A MODERN BROWSER

Step-by-step: What actually happens when you open chatgpt.com

0 HTML + CSS RECEIVED — Browser receives HTML + CSS

```
<div class="card">
  Hello Hitesh
</div>
```

```
.card {
  width: 300px;
  height: 200px;
  background: blue;
  color: white;
  transform: translateX(0px);
}
```

1 DOM CREATED — Browser creates DOM Tree: Document → div.card → "Hello Hitesh"

2 CSS APPLIED — Browser calculates computed styles

```
div.card
width = 300px
height = 200px
background = blue
color = white
```

3 RENDER TREE — Browser builds Render Tree (only visible elements): Render Tree → div.card → "Hello Hitesh"

4 LAYOUT (GEOMETRY) — Browser calculates exact position & size

```
div.card
x = 100
y = 200
width = 300
height = 200
```

5 PAINT (INSTRUCTIONS) — Browser creates paint instructions

1. Draw blue rectangle
2. Draw white text
3. Apply border radius
4. Apply shadows (if any)

...

(Still not pixels)

NOW GPU PIPELINE STARTS

Browser sends visual instructions to GPU

6 LAYER CREATION — Browser promotes elements to layers (when needed)

- Layer 1 Background
- Layer 2 Navbar
- Layer 3 .card (blue box)
- Layer 4 Popup / Other
- …

7 GPU RASTERIZATION — GPU converts vector / shapes / text into pixels (rasterization)

Draw blue rectangle (300 x 200) →

Draw white text "Hello Hitesh" → Hello Hitesh

```
Every pixel gets a color (RGBA)
Example:
Pixel 1 = blue
Pixel 2 = blue
Pixel 3 = white
Pixel 4 = blue
...
For 300 x 200 box = 60,000 pixels
```

8 GPU COMPOSITING — GPU combines all layers like Photoshop layers

Hello Hitesh — Final composed frame (before display)

9 FRAME TO SCREEN — Final pixels sent to monitor

Hello Hitesh — Monitor displays final image

ANIMATION EXAMPLE: `transform: translateX(500px); transition: 1s;` (60 FPS)

WITHOUT GPU ACCELERATION — CPU has to:

- Recalculate layout
- Repaint everything
- Redraw everything

Recalculate Layout → Repaint + Redraw → Frame N+1

Very expensive!

High CPU usage, janky animation

VS

WITH GPU ACCELERATION — Browser says: "GPU, just move this layer." → GPU moves the layer efficiently (no layout/repaint needed)

Frame 1 x = 0 → Frame 2 x = 10 → Frame 3 x = 20 → … Frame 60 x = 500

Smooth 60 FPS animation (Efficient layer movement by GPU)

WHAT GPU DOES DURING ANIMATION

- Creates 60 frames in 1 second
- Each frame: layer at new position
- Only pixels for moved layer updated
- Runs highly parallel on thousands of GPU cores

GPU CORES (Parallel Work): Core 1 (top pixels) · Core 2 (middle pixels) · … · Core N (bottom pixels)

Thousands of cores work together

THE COMPLETE RENDERING PIPELINE (OVERVIEW)

HTML → DOM → CSSOM → Render Tree → Layout (Geometry) → Paint (Instructions) → Layers → GPU Rasterization → Compositing → Final Pixels → Monitor

CPU / Browser Engine | GPU Pipeline

WHAT IS GPU RASTERIZATION?

Converting shapes, text, vectors into actual pixels.

Example: Instruction: Draw blue rectangle → After Rasterization:

WHAT IS GPU COMPOSITING?

Combining multiple layers into final image.

Example Layers:

- Background
- Navbar
- Card (blue box)
- Popup
- Video
- Animation

GPU blends all layers into one final frame.

REAL LIFE: WHAT GPU IS USED FOR ON CHATGPT.COM

- ✓ Text rendering
- ✓ Scrolling
- ✓ Cursor blinking
- ✓ Animations
- ✓ Smooth transitions
- ✓ Shadows & blur
- ✓ Opacity changes
- ✓ Layer compositing
- ✓ Images & icons
- ✓ Video (if any)

… and many more

WHY WE NEED GPU?

- Millions of pixels
- 60+ frames per second
- Parallel processing
- Offloads heavy work from CPU
- Smooth, responsive UI

HOW BROWSER USES GPU – COMPLETE INTERNAL FLOW

THE BIG PICTURE

Browser (Website) → **GPU Process** Secure Broker, Validator, Manager, Isolation Boundary → **Graphics API** Standard Language (OpenGL / Vulkan / DirectX / Metal) → **GPU Driver** Translator + Controller, Manages Hardware, Schedules, Memory, Commands → **GPU Hardware** Actual Physical Chip, Parallel Compute Machine, Renders Pixels → **Monitor (Screen)** Pixels Displayed On Screen

Browser sends commands → GPU Process validates & manages → API standardizes → Driver translates & controls → GPU Hardware executes → Pixels on screen

1 GPU HARDWARE — WHAT IS IT?

GPU is an actual physical chip.

Examples:

- NVIDIA RTX
- AMD Radeon
- Intel Integrated GPU
- Apple GPU

INSIDE GPU HARDWARE:

- Thousands of compute cores
- VRAM controllers
- Command processors
- Texture units
- Raster engines
- Video decoders
- Schedulers

WHAT DOES GPU DO? Performs electrical operations:

- ✓ Fill pixels
- ✓ Blend colors
- ✓ Run shaders
- ✓ Decode video
- ✓ Move textures
- ✓ Calculate lighting

IMPORTANT: GPU hardware does NOT understand JavaScript, HTML, CSS. It only understands very low level commands.

2 GPU DRIVER — WHAT IS IT?

GPU Driver = translator + controller + manager between OS and GPU.

WHAT DRIVER DOES:

1. Translates Commands — Converts API calls into actual GPU instruction packets.
2. Manages VRAM — Manages textures, buffers, memory allocations.
3. GPU Scheduling — Coordinates multiple apps (YouTube, games, browser etc.) for GPU time.
4. Hardware Control — Directly accesses registers, controls DMA, handles interrupts. Runs at kernel level (very close to system).

WHY DRIVER IS DANGEROUS? Because it runs with kernel-level privileges. Bugs can crash the system or cause instability.

3 GRAPHICS API — WHAT IS IT?

Graphics API = standard language / interface between browser and driver.

WHY API NEEDED?

- Different GPU companies (NVIDIA, AMD, Intel, Apple) have different hardware.
- Browser cannot support all directly.
- API provides a common contract / language.

EXAMPLES: OpenGL · Vulkan · DirectX · Metal

HOW IT WORKS: Browser says (using API): "Draw triangle"

Driver converts API calls into hardware-specific instructions.

4 GPU PROCESS — WHAT IS IT?

GPU Process is a software process inside browser.

ACTS AS:

- ✓ Broker
- ✓ Manager
- ✓ Security Boundary
- ✓ Validator
- ✓ Scheduler

WHY GPU PROCESS NEEDED?

- Renderer runs untrusted website code (JS, WebGL, shaders).
- Direct access to GPU = very risky (could crash driver, exploit bugs, corrupt memory).
- GPU Process sits in the middle as a secure gate.

ACTUAL WORK OF GPU PROCESS:

1. Validation — Checks commands, textures, operations for safety.
2. Isolation — If GPU crashes, only GPU process dies, browser survives.
3. Resource Management — Manages textures, GPU memory, compositing, rendering queues.
4. Security Boundary — Renderer cannot directly touch driver, hardware, or kernel.

FULL INTERNAL FLOW STEP-BY-STEP

- STEP 1 — Renderer says: "Need blue rectangle."
- STEP 2 — GPU Process validates request.
- STEP 3 — Graphics API receives standardized commands (ex: OpenGL draw).
- STEP 4 — Driver converts API calls into hardware-specific electrical instructions.
- STEP 5 — GPU hardware executes massively parallel computations.
- STEP 6 — Pixels generated.
- STEP 7 — Monitor displays final image.

KEY CONCEPTS (TERMS EXPLAINED)

Term	Explanation
GPU (Graphics Processing Unit)	A specialized processor designed for parallel computations, mainly used for rendering graphics, AI, video, etc.
Compute Core	Small processing units inside GPU that perform mathematical operations in parallel.
VRAM (Video RAM)	High-speed memory on GPU used to store textures, frames, buffers, shaders, etc.
Shader	Small programs written in GPU language that run on GPU for effects like lighting, shadows.
API (Application Programming Interface)	A standard set of rules & functions that applications use to communicate with GPU driver.
Driver	Software that translates API commands into hardware instructions and controls GPU.
DMA (Direct Memory Access)	Allows GPU to read/write memory directly without CPU involvement (very high speed).
Command Buffer	A list of GPU commands stored in memory and executed by GPU in sequence.

GPU CRASH – WHAT DOES IT ACTUALLY MEAN?

MOSTLY, HERE WHEN WE SAY:

GPU crash means: GPU Process OR GPU Driver crash.

NOT necessarily: actual physical GPU hardware damage

3 DIFFERENT THINGS – IMPORTANT DISTINCTION

1 GPU PROCESS CRASH

This is: browser ka software process.
Example: Chrome GPU Process
This is: software, NOT hardware.

WHAT HAPPENS?

- bad command
- memory corruption
- IPC issue
- invalid shader

Then: GPU Process dies
BUT: actual GPU hardware alive hota hai.

EXAMPLE FLOW: Renderer ↓ GPU Process ✗ crash ↓ Driver ↓ GPU Hardware ✓ alive

BROWSER USUALLY RECOVERS — New GPU process create ho jata hai.

2 GPU DRIVER CRASH

THIS is more serious.
Driver: software bridge between: OS, browser, GPU hardware.

DRIVER CRASH MEANS: GPU communication layer crash.

EXAMPLE FLOW: Renderer ↓ GPU Process ↓ GPU Driver ✗ crash ↓ GPU Hardware

RESULT (POSSIBLE):

- browser freeze
- screen flicker
- black screen
- driver reset
- BSOD sometimes

HARDWARE STILL OFTEN FINE HOTA HAI. Problem control software me hoti hai.

3 ACTUAL GPU HARDWARE CRASH / HANG

THIS is actual hardware-level issue. Rare.

EXAMPLES:

- overheating
- faulty VRAM
- electrical issue
- hardware instability

Then: actual GPU chip stop responding

EXAMPLE: GPU Hardware ✗ hang

NOW (POSSIBLE):

- screen freeze
- artifacts
- system instability

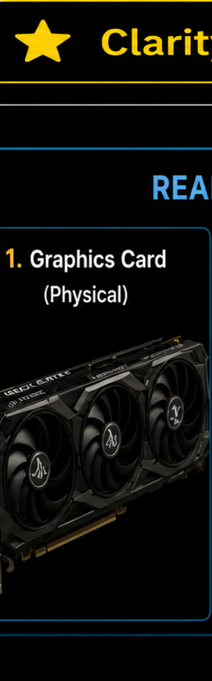
THIS IS REAL HARDWARE PROBLEM.

WHY THIS DISTINCTION MATTERS?

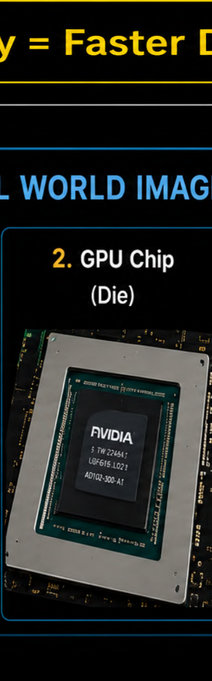
- ✓ Helps identify the real cause of crash/hang.
- ✓ Different fixes for software vs driver vs hardware.
- ✓ Saves time in debugging and troubleshocting.
- ✓ Prevents unnecessary hardware replacement.
- ✓ Know whether system is recoverable or not.
- ✓ Understanding internals makes you a better developer.
- ✓ Helps in performance tuning and stability.

★ Clarity = Faster Debugging = Better Solutions

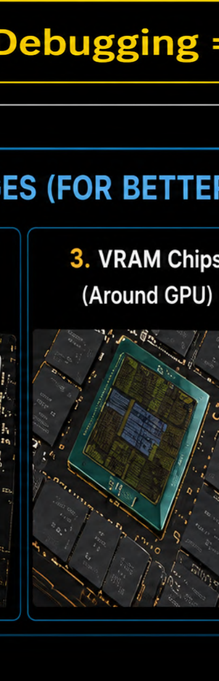
REAL WORLD IMAGES (FOR BETTER UNDERSTANDING)



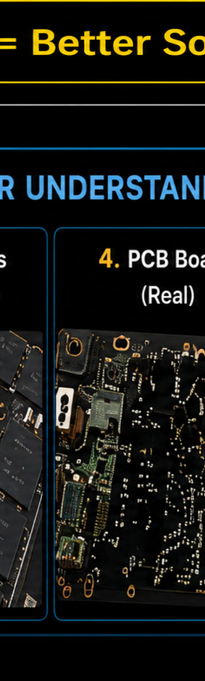
1. Graphics Card (Physical)



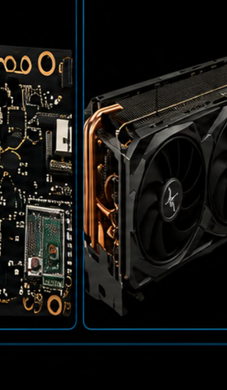
2. GPU Chip (Die)



3. VRAM Chips (Around GPU)



4. PCB Board (Real)



5. Cooling System

GLOSSARY (QUICK RECAP)

Term	Meaning
GPU Hardware	Actual parallel computing chip.
GPU Driver	Low-level translator/controller.
Graphics API	Standard language/interface.
GPU Process	Browser-side secure broker.
VRAM	Video RAM (GPU memory).
IPC	Inter Process Communication.